통합 구현

Fullstack 풀스택 개발자 과정

1조: 박한결, 박영준, 원병선

CONTENTS

1

구동 확인



2

SecurityConfig
LinkController



3

PostRepository
LinkServiceImpl



4

PostDTO
LinkScheduler
LinkServiceTest



1. 구동확인 – MySQL

```sql
-- S-Link 연계 데이터 스키마 (MySQL testdb / localhost:3306 / root / 1234)
-- 외부 게시글(Post)을 수신·저장. id 는 외부 시스템의 식별자를 그대로 사용(upsert).
DROP TABLE IF EXISTS tbl_post;

CREATE TABLE tbl_post (
    id        BIGINT       NOT NULL,          -- 외부 시스템 식별자(PK, 자동증가 아님)
    user_id   BIGINT,
    title     VARCHAR(500),
    body      TEXT,
    create_at TIMESTAMP,
    PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

user_id	title	body	create_at
NULL	NULL	NULL	NULL

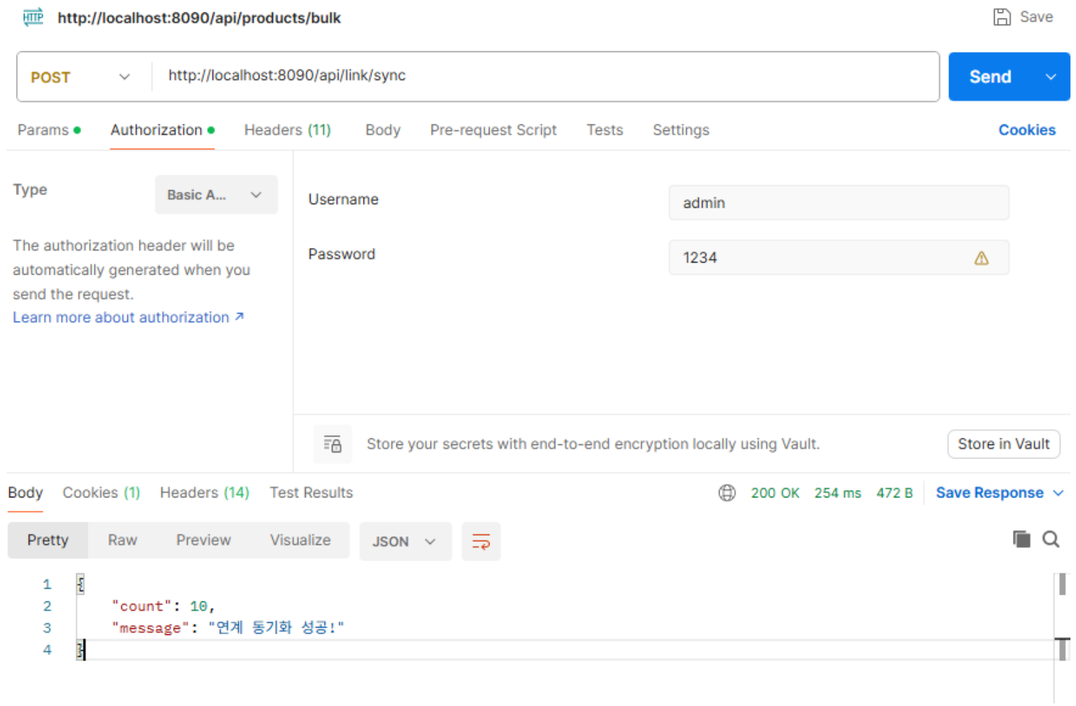
st 1

Action Output

	Time	Action	Message
1	10:22:21	DROP TABLE IF EXISTS tbl_post	0 row(s) affected, 1 warning(s): 105
2	10:22:21	CREATE TABLE tbl_post (id BIGINT NOT NULL, -- 외부 시스템 식별자(PK, 자동증가 아...	0 row(s) affected
3	10:22:36	SELECT * FROM testdb.tbl_post	0 row(s) returned
4	10:22:44	SELECT * FROM testdb.tbl_post	0 row(s) returned

MySQL 테이블 생성 & 내용 작성

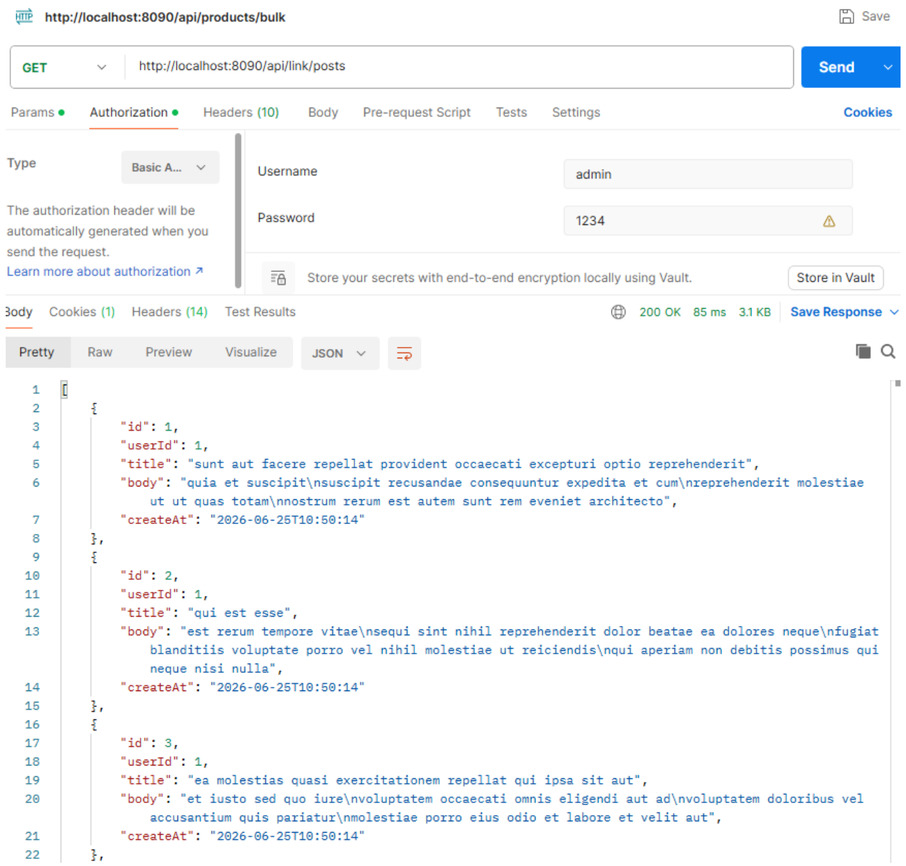
1. 구동확인 – 연계동기화



수동 동기화(연계 수신)

외부 API 데이터를 DB에 수동으로 저장하기 위해,
접근 권한이 있는 ADMIN 사용자로 로그인한 후 데이터 동기화
(POST /api/link/sync)를 수행한다.

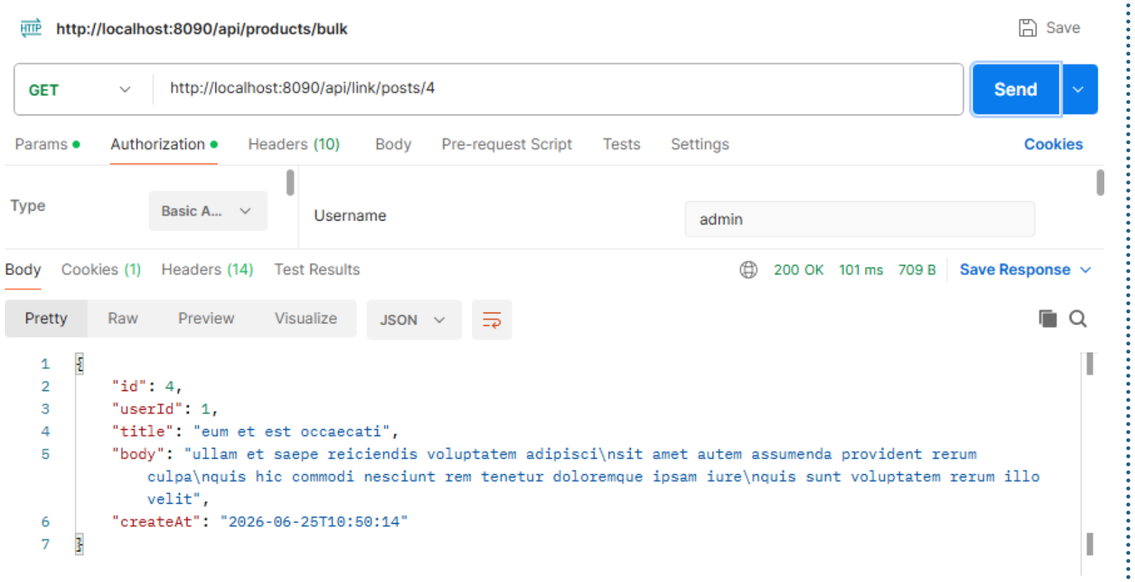
1. 구동확인 – 재제공



연계 데이터 목록 재제공

전체 연계 데이터를 조회하기 위해, 로그인한 사용자라면 누구나 접근 가능한 권한을 기반으로 (GET /api/link/posts) 를 통해 데이터를 조회한다

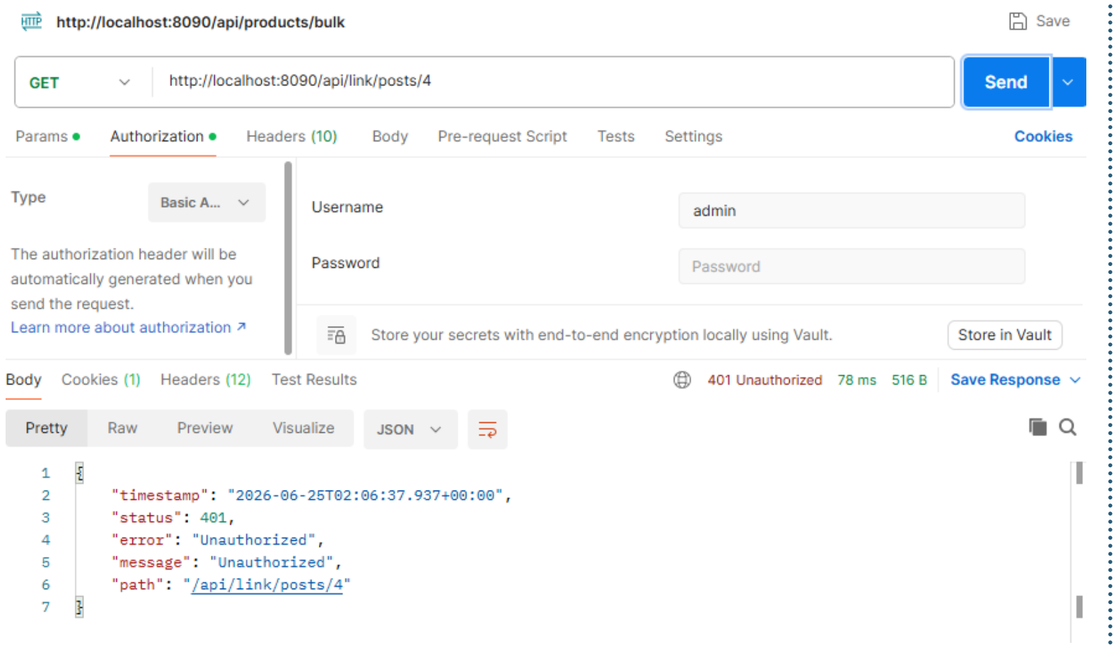
1. 구동확인 – 단건 재제공



단건 연계 데이터 재제공

단건 연계 데이터를 조회하기 위해, 로그인한 사용자라면 누구나 접근 가능한 권한을 기반으로 (GET /api/link/posts/{id}) 를 통해 데이터를 조회한다

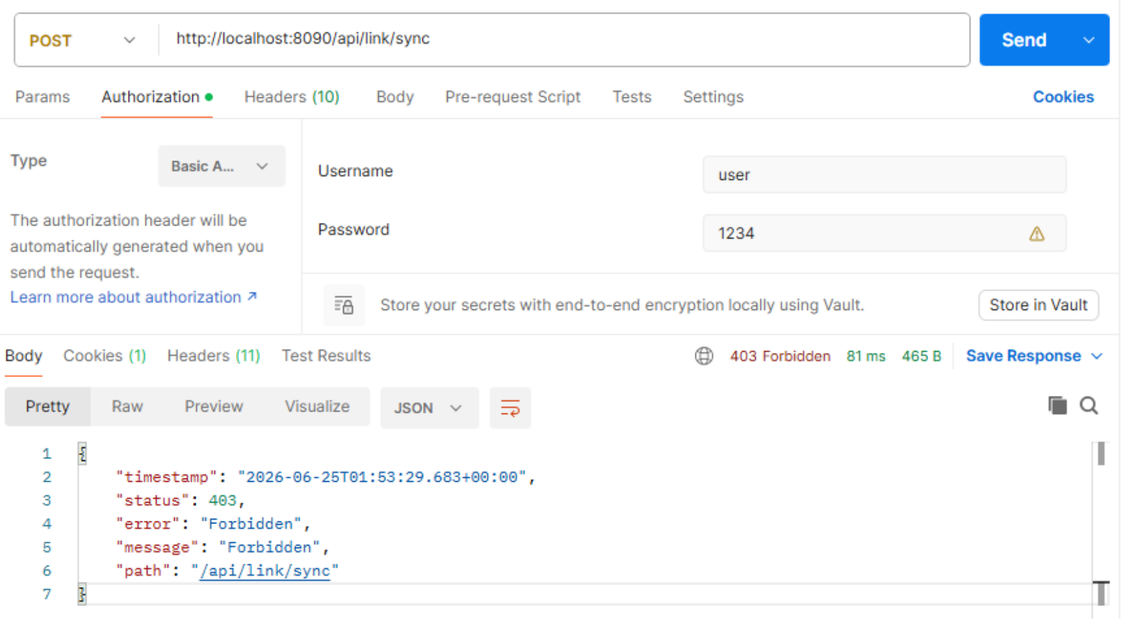
1. 구동확인 – 보안(401)



401 Unauthorized 인증 오류

로그인(인증)을 하지 않고 요청했거나, 잘못된 아이디/비밀번호를 입력한 경우 누구인지 증명하지 못해(미인증) 접근이 거부되고 오류가 발생한다.

1. 구동확인 – 보안(403)

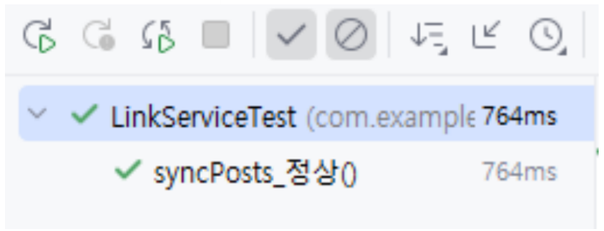


403 Forbidden 인가 오류

로그인은 완료되었으나 일반 사용자(USER) 계정으로 관리자(ADMIN) 전용 메뉴에 접근한 경우, 해당 요청에 대한 인가가 없어 거부되고 오류가 발생한다.

1. 구동확인 – 배치 동기화&테스트

```
2026-06-25T11:11:18.437+09:00  INFO 12560 --- [demo] [   scheduling-1] c.example.demo.Scheduled.LinkScheduler    : [Link] 배치 동기화 완료 - 10건
2026-06-25T11:12:18.613+09:00  INFO 12560 --- [demo] [   scheduling-1] c.example.demo.Scheduled.LinkScheduler    : [Link] 배치 동기화 완료 - 10건
```



스케줄러 배치 동기화

LinkScheduler 메서드의 @Scheduled가 정해진 주기(60초)마다 LinkService.syncPosts()를 자동 호출해 동기화를 수행하고 로그를 남긴다.

연계 동기화 테스트

테스트코드를 실행해 저장 건수가 0보다 큰지, 조회한 전체 목록이 비어있지 않은지 검증한다.

1. 구동확인 – COMMIT

```
579b023 (HEAD -> dev, origin/dev) V1.0 [조장] 최종 구동 확인 완료
2f05c76 V0.3 Merge pull request #6 from Fullstack-Tests/feature/원병선
a381a03 V0.2 Merge pull request #4 from Fullstack-Tests/feature/박영준
29936c6 V0.1Merge pull request #2 from Fullstack-Tests/feature/박한결
29b3886 (origin/feature/원병선) 원병선 - findById(id) (없으면 MyBizException) → PostDTO.from
e9b483d 원병선 - @transactional(readOnly = true)
376bf68 원병선 - findAll() → PostDTO.from 으로 매핑하여 List 반환
95fe79b 원병선 - @transactional(readOnly = true)
af0a2c0 원병선 - 저장 건수(int) 반환
d46f095 원병선 - 응답 PostDTO[] 를 돌며 createAt=now() 설정 후 postRepository.save(dto.toEntity())
30d6757 원병선 - 상태코드가 HttpStatus.OK 가 아니거나 body 가 null 이면 MyBizException 발생 (연계 장애) (수정)
7821068 원병선 - 응답 PostDTO[] 를 돌며 createAt=now() 설정 후 postRepository.save(dto.toEntity())
5b36fa1 원병선 - RestTemplate + UriComponentsBuilder 로 (apiBase + "/posts", ?_limit=10) GET 호출
c6eddae 원병선 - @transactional 적용
ea30ca5 (origin/feature/박영준) 박영준 - 단건 재제공 - GET /api/link/posts/{id}
6a66d9c 박영준 - 연계 데이터 목록 재제공 - GET /api/link/posts
7ff96a8 박영준 - 수동 동기화(연계 수신) - POST /api/link/sync
1ff4d9d 박영준 - user / admin, 비밀번호 "1234"로 인메모리 사용자 구현
d68f904 박영준 - httpBasic 사용으로 인증처리
e19f228 박영준 - POST /api/link/sync 인가처리
3a557c7 박영준 - csrf 비활성화
359b24a (origin/feature/박한결, feature/박한결) 박한결 - 전체 목록 검증
febd25f 박한결 - 저장 건수 검증
2a15f6a 박한결 - 스케줄러 완성
6eb2358 박한결 - Entity → DTO 변환
e660a82 박한결 - DTO → Entity 변환
29a9618 (origin/main, origin/HEAD, main) [조장] 베이스 업로드
6df67a2 Initial commit
```

commit 확인

2. SecurityConfig

```java
    @Bean  pyj-0313 +1
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf( CsrfConfigurer<HttpSecurity> csrf -> {csrf.disable();}); //CSRF 비활성화
        //인가
        http.authorizeHttpRequests(( AuthorizationManagerRequestMat... auth) -> {
            auth.requestMatchers(HttpMethod.POST,"/api/link/sync").hasRole("ADMIN"); // "ADMIN" 권한이 있는자만 접근가능
            auth.requestMatchers("/api/link/**").authenticated(); // 로그인한 사람 누구나 허용
            auth.anyRequest().permitAll(); // 그외
    });
        //인증
        http.httpBasic(Customizer.withDefaults()); // httpBasic 사용
        return http.build();
    }

    @Bean  1gyeolpark
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    // TODO: 인메모리 사용자 – user(USER) / admin(ADMIN), 비밀번호 "1234"
    @Bean  pyj-0313 +1
    public InMemoryUserDetailsManager userDetailsManager() {
        UserDetails user = User.withUsername("user") // 인메모리 사용자
                .password(passwordEncoder().encode( rawPassword: "1234")).roles("USER").build();
        UserDetails admin = User.withUsername("admin") // 인메모리 사용자
                .password(passwordEncoder().encode( rawPassword: "1234")).roles("ADMIN").build();
        return new InMemoryUserDetailsManager(user, admin);
    }
}
```

연계 접근 보안 규칙 구현

CSRF를 비활성화하고, 인가처리로 POST 방식의 /api/link/sync는 ADMIN 권한을 가진 사용자만 접근 가능하도록 제한하고, /api/link/ 하위 URL은 로그인한 사용자라면 누구나 허용하고, 나머지는 모두 허용한다. 인증 방식은 httpBasic을 사용한다.

인메모리 사용자

비밀번호 암호화 방식으로 BCrypt를 사용하고, 인메모리 사용자로 USER 권한의 user와 ADMIN 권한의 admin을 각각 비밀번호 "1234"로 등록한다.

2. LinkController

```
    @PostMapping(value = "/sync", produces = MediaType.APPLICATION_JSON_VALUE)  pyj-0313
    public ResponseEntity<Map<String, Object>> sync()
    {
        Map<String, Object> responseMap = new HashMap<>(); // 데이터를 담을 Map 생성
        int count = linkService.syncPosts(); // 동기화 후 저장 건수 반환
        responseMap.put("count",count); // 저장 건수 Map에 추가
        responseMap.put("message","연계 동기화 성공!"); // 메시지 Map에 추가
        return ResponseEntity.status(HttpStatus.OK).body(responseMap); // 200 + Map 반환
    }

    // TODO: 연계 데이터 목록 재제공 – GET /api/link/posts
    //  - linkService.getPosts() 결과를 200 반환
    @GetMapping(value = "/posts", produces = MediaType.APPLICATION_JSON_VALUE)  pyj-0313
    public ResponseEntity<?> posts() // 반환 타입 미정
    {
        return ResponseEntity.ok(linkService.getPosts()); // 전체 목록 조회 후 200 반환
    }


    // TODO: 단건 재제공 – GET /api/link/posts/{id}
    //  - linkService.getPost(id) 결과를 200 반환
    @GetMapping(value = "/posts/{id}", produces = MediaType.APPLICATION_JSON_VALUE)  pyj-0313
    public ResponseEntity<PostDTO> post(@PathVariable("id") Long id)
    {
        return ResponseEntity.ok(linkService.getPost(id)); // id로 단건 조회 후 200 반환
    }
}
```

수동 동기화

POST방식으로 /api/link/sync 요청이 들어오면 syncPosts()를 호출해 외부 api를 동기화하고, 저장 건수(count)와 성공 메시지를 Map에 담아 200으로 반환한다.

연계 데이터 목록 재제공

GET방식으로 /api/link/posts 요청이 들어오면 getPosts()를 호출해 전체 목록을 조회하고 200으로 반환한다.

단건 재제공

GET방식으로 /api/link/posts/{id} 요청이 들어오면 URL에서 id를 추출해 getPost(id)를 호출하고 단건 조회 결과를 200으로 반환한다.

3. PostRepository

```
// TODO: JpaRepository<Post, Long> 를 상속하도록 선언하라.
//  (기본 save/findAll/findById/deleteById 등을 자동 제공받는다)
@Repository  2개 사용 위치
public interface PostRepository extends JpaRepository<Post, Long> {
}
```

상속 선언

JpaRepository<Post, Long> 를 상속하도록 선언

3. LinkServiceImpl

```java
@Override  0개의 사용위치
@Transactional
public int syncPosts() {
    RestTemplate restTemplate = new RestTemplate();

    URI uri = UriComponentsBuilder.fromHttpUrl(apiBase) UriComponentsBuilder
            .path("/posts")
            .queryParam( name: "_limit",  ...values: 10)
            .build() UriComponents
            .toUri();

    ResponseEntity<PostDTO[]> responseEntity = restTemplate.exchange(
            uri,
            HttpMethod.GET,
             requestEntity: null,
            PostDTO[].class
    );
```

@Transactional 적용

Db 작업의 원자성을 보장하기 위해서 선언

RestTemplate + UriComponentsBuilder 로 GET 호출

외부 API 주소를 안전하게 조립한 후에 URI 객체로 변환
(기본 주소 + /posts + ?_limit=10)

지정한 URI로 HTTP GET 요청을 보내고, 응답 데이터를
POST[] 배열 형태로 보여줌
uri = 호출할 외부 API 주소
HttpMethod.GET = HTTP 요청 메서드(조회)
null = 요청 메세지 바디 (GET이라서 없음)
PostDTO[].class = 응답 바디를 변환할 클래스 타입

3. LinkServiceImpl

```
if (responseEntity.getStatusCode() != HttpStatus.OK || responseEntity.getBody() == null) {
    log.error("외부 API 연계 장애 발생. 상태코드: {}", responseEntity.getStatusCode());
    throw new MyBizException("연계 장애 발생");
}
```

MyBizException 발생

상태코드가 HttpStatus.OK 가 아니거나 body 가 null 이면
MyBizException 실행

콘솔/로그에 에러 내용과 실패 상태 코드를 기록하고
MyBizException를 강제로 발생시켜 메서드를 중단하고
트랜잭션을 롤백 시킴

3. LinkServiceImpl

```
PostDTO[] postArray = responseEntity.getBody();


List<Post> entityList = Arrays.stream(postArray) Stream<PostDTO>
        .map( PostDTO dto -> {
            Post entity = dto.toEntity();
            entity.setCreateAt(LocalDateTime.now());
            return entity;
        }) Stream<Post>
        .toList();

postRepository.saveAll(entityList);

int saveCount = entityList.size();
return saveCount;
```

응답 PostDTO[] 를 돌며 createAt=now() 설정

외부 API 응답 본문에서 PostDTO 배열 데이터를 꺼냄

자바 스트림(Stream)을 활용하여 DTO 배경을 가공하고
Entity 리스트로 변환
→ DTO를 DB 저장용 Entity로 변환
→ 엔티티에 현재 시스템 생성 시간 설정
→ 변환된 엔티티 반환

스트림 결과를 List 형태로 수집

변환된 엔티티 리스트를 DB에 한 번에 일괄 저장

최종적으로 DB에 저장된 총 데이터 건수(int)를 구해서 반환

3. LinkServiceImpl

```
@Override  0개의 사용위치
@Transactional(readOnly = true)
public List<PostDTO> getPosts() {
    List<Post> entityList = postRepository.findAll();

    return entityList.stream() Stream<Post>
            .map( Post entity -> PostDTO.from(entity)) Stream<PostDTO>
            .toList();
}
```

findAll() → PostDTO.from 으로 매핑하여 List 반환

@Transaction 메서드를 통해 조회 목적임을 선언

DB의 posts 테이블에 있는 모든 게시글 엔티티를 조회

조회한 엔티티 리스트를 자바 스트림을 사용해 DTO 리스트로 변환하여 반환 ⇒ 각 엔티티를 PostDTO 객체로 변환 → 변환된 DTO들을 최종 List 형태로 묶어서 반환

3. LinkServiceImpl

```java
@Override  0개의 사용위치
@Transactional(readOnly = true)
public PostDTO getPost(Long id) {
    Optional<Post> postOptional = postRepository.findById(id);

    if (postOptional.isPresent()) {
        return PostDTO.from(postOptional.get());
    }

    throw new MyBizException("존재하지 않는 게시글입니다.");
```

findById(id) (없으면 MyBizException)
→ PostDTO.from

@Transaction 메서드를 통해 조회 목적임을 선언

DB에서 ID로 게시글을 조회하며, null 방지를 위해 Optional 을 사용

Optional 상자 안에 조회된 게시글 데이터가 존재하는지 확인하고 데이터가 있다면 상자에서 엔티티를 꺼내(get) DTO 로 변환한 후 반환

데이터가 없다면 예외를 던져 처리를 중단함

4. PostDTO

```java
// DTO -> Entity 변환
// 외부 API에서 받은 DTO 데이터를 DB 저장용 Entity로 변환
public Post toEntity() {  0개의 사용위치
    // PostDTO(DTO)의 값을 Post(Entity) 필드에 매핑해 변환
    Post post = Post.builder()  // builder(): 객체를 단계적으로 생성하는 패턴
            .id(id)
            .userId(userId)
            .title(title)
            .body(body)
            .createAt(createAt)
            .build();

    return post;
}

// TODO: Entity → DTO 변환(static). PostDTO.builder() 로 매핑하여 반환

// Entity -> DTO 변환
// DB에 저장된 Entity를 외부 API 응답용 DTO로 변환
public static PostDTO from(Post p) { // static: 객체 생성 없이 바로 호출하기 위해 사용
    // Post(Entity)의 값을 PostDTO(DTO) 필드에 매핑해 변환
    PostDTO postDTO = PostDTO.builder()
            .id(p.getId())
            .userId(p.getUserId())
            .title(p.getTitle())
            .body(p.getBody())
            .createAt(p.getCreateAt())
            .build();

    return postDTO;
}
```

DTO → Entity 변환

외부에서 받은 DTO를 DB에 저장하기 위해 Entity로 변환하는 메서드.
builder()로 PostDTO 내부 값을 Post 필드에 각각 매핑하여 반환한다.

Entity → DTO 변환

DB에 저장된 Entity를 외부 응답하기 위해 DTO로 변환하는 메서드.
builder()로 Post 값을 PostDTO 필드에 각각 매핑하여 반환한다.

4. LinkScheduler

```java
// 지정 주기마다 자동 실행 설정.
// @Scheduled: 스프링에서 정해진 주기마다 메서드를 자동으로 실행하게 하는 어노테이션
// fixedDelayString: 이전 작업이 끝난 시점부터 일정한 지연 시간(밀리초 단위)을 두고 다음 작업을 실행하도록 지정
@Scheduled(fixedDelayString = "${link.sync.delay:60000}")
public void scheduledSync() {
    // TODO: @Scheduled + syncPosts 호출 구현
    try{
        // syncPosts(): DB 저장, 저장 건수 반환
        // syncPosts()를 호출해 연계 동기화를 수행하고, 저장된 총 건수(n)을 반환받는다
        int n = linkService.syncPosts();
        log.info("[Link] 배치 동기화 완료 - " + n + "건");
    }catch(Exception e) {
        // try-catch로 예외 발생 시 멈추지 않도록 감싸고 에러 로그를 남긴다
        log.error("[Link] 배치 동기화 실패: " + e.getMessage());
    }

}
```

스케줄러 주기적 자동 실행

60초마다 자동으로 동기화가 실행되며, 예외 발생 시 에러로그를 남기는 메서드.

@Scheduled: 스프링에서 정해진 주기마다 메서드를 자동으로 실행하게 하는 어노테이션
fixedDelayString: 이전 작업이 끝난 시점부터 일정한 지연 시간(밀리초 단위)을 두고 다음 작업을 실행하도록 지정
syncPosts(): DB 저장, 저장 건수 반환
syncPosts()를 호출해 연계 동기화를 수행하고, 저장된 총 건수(n)을 반환받는다.
try-catch로 예외 발생 시 멈추지 않도록 감싸고 에러 로그를 남긴다.

4. LinkServiceTest

```java
@Test
void syncPosts_정상() {
    // syncPosts()를 호출해 데이터를 DB에 저장하고, 반환된 저장 건수 count에 저장
    int count = linkService.syncPosts();
    // syncPosts(): DB 저장, 저장 건수 반환
    // 반환된 저장 건수 count가 0보다 큰지 검증
    assertTrue( condition: count > 0,  message: "저장 건수는 0 보다 커야 합니다.");
    // getPosts(): DB 저장된 연계 데이터 목록 제공
    // getPosts().isEmpty()를 호출해 조회한 전체 목록이 비어있지 않은지 검증
    assertTrue(!linkService.getPosts().isEmpty(),  message: "저장된 데이터가 존재하지 않습니다.");
}
```

저장 건수 검증

반환된 저장 건수가 0보다 큰지 검증한다.
syncPosts()를 호출해 데이터를 DB에 저장하고, 반환된 저장 건수 count에 저장한다.
반환된 저장 건수 count가 0보다 큰지 검증한다.

전체 목록 검증

조회된 전체 목록이 비어있지 않은지 검증한다.
getPosts().isEmpty()를 호출해 조회한 전체 목록이 비어있지 않은지 검증한다.



감사합니다

Fullstack